Київський національний університет імені Тараса Шевченка

Факультет Інформаційних Технологій

Кафедра мережевих і Інтернет технологій

Лабораторна робота №4

по дисципліні

«Технології створення програмних продуктів»

Шаблони “Composite”, “Decorator”

Група: МІТм-11

Студент: Соломєнцев Н. А.

Київ 2026

Створіть ієрархію класів для графічного редактора, де Shape є базовим інтерфейсом, а CompositeShape може містити кілька об’єктів Shape.

```
CompositeShape.cs
src > CompositeProject > CompositeShape.cs
namespace CompositeProject;

using System.Collections.Generic;


public class CompositeShape: IShape {
    private List<IShape> _shapes = new List<IShape>();
    private string _name;

    public CompositeShape(string name, List<IShape> shapes) {
        _name = name;
        _shapes.AddRange(shapes);
    }

    public List<string> GetPrimitivesNames() {
        List<string> primitives = new List<string> {_name};
        foreach (IShape shape in _shapes) {
            primitives.AddRange(shape.GetPrimitivesNames());
        }
        return primitives;
    }
}
```

```
Shape.cs
src > CompositeProject > Shape.cs
namespace CompositeProject;

using System.Collections.Generic;


public class Shape: IShape {
    private string _name;

    public Shape(string name) {
        _name = name;
    }

    public List<string> GetPrimitivesNames() {
        return new List<string> {_name};
    }
}
```

```
IShape.cs
src > CompositeProject > IShape.cs
namespace CompositeProject;

using System.Collections.Generic;

public interface IShape {
    public abstract List<string> GetPrimitivesNames();
}
```

рис. 1.: Фрагмент коду для реалізації шаблону “Composite”

Реалізуйте клас IceCreamDecorator, який додасть нові інгредієнти до базового класу IceCream без зміни його коду.

```
IceCreamDecorator.cs M
src > DecoratorProject > IceCreamDecorator.cs
namespace DecoratorProject {
    abstract public class IceCreamDecorator: IIceCream {
        private IIceCream _iceCreamInstance;

        public void SetIceCream(IIceCream iceCream) {
            _iceCreamInstance = iceCream;
        }

        virtual public string GetDescription() {
            return _iceCreamInstance.GetDescription();
        }
    }

    public class CreamyIceCreamDecorator: IceCreamDecorator {
        public override string GetDescription() {
            return "Creamy " + base.GetDescription();
        }
    }

    public class SweetIceCreamDecorator: IceCreamDecorator {
        public override string GetDescription() {
            return "Sweet " + base.GetDescription();
        }
    }
}
```

```
IceCream.cs
src > DecoratorProject > IceCream.cs
using System.Collections.Generic;


namespace DecoratorProject {

    public interface IIceCream {
        public string GetDescription();
    }

    public class IceCream: IIceCream {
        private string _size;

        public IceCream(string size) {
            _size = size;
        }

        public string GetDescription() {
            return $"Ice cream size: {_size}";
        }

    }
}
```

рис. 2.: Фрагмент коду для реалізації шаблону “Decorator”

Висновки:
Шаблон “Composite” використовується у випадках коли виникає потреба взаємодії як з окремим об’єктом так і з групою об’єктів через той самий інтерфейс.
Шаблон “Decorator” використовується у випадках коли треба виконати певні дії з елементами кодової бази, часто “до” чи “після” основного функціоналу. Наприклад, записувати час виконання певної функції - час записується до виклику основного

функціоналу та після. Або виконати ряд одноманітних операцій на результаті обробки основного функціоналу.